

Control Flow Detective

Read each piece of code carefully. Do NOT run it in your head too fast — trace through it line by line, just like the computer would. Write down exactly what would be printed to the console.

Directions: For each problem, write the exact output, including spacing, capitalization, and line breaks. If nothing is printed, write (**nothing**). If the code would cause an error, write **ERROR** and explain why in one sentence.

Worked Example (this one is done for you — study how the tracing works!)

```
1 x = 3
2 if x > 5:
3     print("big")
4 else:
5     print("small")
6 print("done")
```

How to trace it:

1. `x` is set to 3.
2. Check the condition: is `3 > 5`? No, that's false.
3. Since the `if` was false, we jump to the `else` block and run `print("small")`.
4. The `if/else` is finished, so we move to the next line, `print("done")`, which always runs no matter which branch we took.

Output:

```
small
done
```

Quick reminders:

- `%` is the **modulo** operator — it gives you the *remainder* after dividing. For example, `7 % 2` is 1 (7 divided by 2 is 3 with 1 left over), and `10 % 5` is 0 (10 divides evenly by 5, so there's no remainder). A number is **even** if `n % 2 == 0`, and **odd** if `n % 2 == 1`.
- `**` is the **exponent** operator — it means "raised to the power of." For example, `2 ** 3` means 2^3 , which is 8, and `5 ** 2` means 5^2 , which is 25.

1 Conditionals

Problem 1

```
1 x = 7
2 if x > 5:
3     print("big")
4 else:
5     print("small")
```

Output:

Problem 2

```
1 temperature = 68
2
3 if temperature > 80:
4     print("It's hot!")
5 elif temperature > 60:
6     print("Nice day.")
7 else:
8     print("Bring a jacket.")
```

Output:**Problem 3**

```
1 grade = 72
2
3 if grade >= 90:
4     print("A")
5 elif grade >= 80:
6     print("B")
7 elif grade >= 70:
8     print("C")
9 else:
10    print("F")
```

Output:**Problem 4 (tricky!)**

```
1 password = "wizard123"
2
3 if password == "wizard123":
4     print("Access granted")
5 if password == "Wizard123":
6     print("Access granted (capital W)")
7 else:
8     print("Access denied (capital W)")
```

Output:

2 Loops

A quick note on break and continue:

Sometimes you want to change how a loop behaves *while it's running*, instead of just letting it finish normally. Two keywords let you do that:

- **break** — immediately **stops the loop completely**. The program jumps to whatever comes after the loop, even if there were more values left to go through.
- **continue** — **skips the rest of the current pass** through the loop and jumps straight to the next one. It does NOT stop the loop — it just skips ahead.

```
1 for i in range(5):
2     if i == 3:
3         break
4     print(i)
```

This prints 0 1 2 and then stops — once `i` hits 3, the loop ends completely.

```
1 for i in range(5):
2     if i == 3:
3         continue
4     print(i)
```

This prints 0 1 2 4 — when `i` is 3, that one pass is skipped, but the loop keeps going afterward.

Problem 5

```
1 for i in range(5):
2     print(i)
```

Output:

Problem 6

```
1 count = 0
2 while count < 3:
3     print("Hello!")
4     count = count + 1
```

Output:

Problem 7

```
1 total = 0
2 for num in range(1, 5):
3     total = total + num
4 print(total)
```

Output:

Problem 8 (break and continue)

```
1  for i in range(10):  
2      if i == 3:  
3          continue  
4      if i == 6:  
5          break  
6      print(i)
```

Output:

Problem 9 (nested loop)

```
1  for i in range(3):  
2      for j in range(2):  
3          print(i, j)
```

Output:

3 Functions

Problem 10

```
1 def greet(name):  
2     print("Hi, " + name + "!")  
3  
4 greet("Maryam")  
5 greet("Amina")
```

Output:

Problem 11

```
1 def square(x):  
2     return x * x  
3  
4 result = square(4)  
5 print(result)
```

Output:

Problem 12 (return vs. print — read carefully!)

```
1 def mystery(n):  
2     if n % 2 == 0:  
3         return "even"  
4     else:  
5         return "odd"  
6  
7 print(mystery(7))  
8 print(mystery(10))
```

Output:

Problem 13 (default arguments)

```
1 def power(base, exponent=2):  
2     return base ** exponent  
3  
4 print(power(3))  
5 print(power(2, 3))
```

Output:

4 Combining Everything

Problem 14

```
1 def check_number(n):
2     if n < 0:
3         return "negative"
4     elif n == 0:
5         return "zero"
6     else:
7         return "positive"
8
9 numbers = [-3, 0, 8, -1]
10 for num in numbers:
11     print(check_number(num))
```

Output:

Problem 15

```
1 def countdown(n):
2     while n > 0:
3         print(n)
4         n = n - 1
5     print("Liftoff!")
6
7 countdown(3)
```

Output:

Problem 16 (challenge: functions calling functions)

```
1 def double(x):
2     return x * 2
3
4 def triple(x):
5     return x * 3
6
7 def combo(x):
8     if x % 2 == 0:
9         return double(x)
10    else:
11        return triple(x)
12
13 for i in range(1, 5):
14     print(combo(i))
```

Output:

Problem 17 (challenge: nested loop + conditional)

```
1 for i in range(1, 4):
2     for j in range(1, 4):
3         if (i + j) % 2 == 0:
4             print(str(i) + "+" + str(j) + " = even sum")
```

Output:**Problem 18 (challenge: FizzBuzz)**

```
1 for n in range(1, 16):
2     if n % 15 == 0:
3         print("FizzBuzz")
4     elif n % 3 == 0:
5         print("Fizz")
6     elif n % 5 == 0:
7         print("Buzz")
8     else:
9         print(n)
```

Output: